

Statement 3

5)TEST CASES AND VERIFICATION RESULT:

This section describes how the ATM simulator is systematically tested to ensure the software works correctly, catches user errors, and remains stable under different conditions.

5.1)Valid Input Testing (Happy Path Execution):

This test verifies that the program performs flawlessly when a user enters all information correctly. It tests the "ideal scenario": entering the correct PIN on the first try, successfully checking the balance, depositing a normal amount of cash, and withdrawing an amount well within the account balance. The goal is to confirm that variables update accurately and the UI routing functions as intended.

5.2) Invalid Input and Attempt-Limit Interception:

This test checks how the program handles incorrect human behavior on security screens or menu selections. Testers intentionally enter the wrong PIN multiple times to verify that the attempt counter decreases properly and that the application safely locks the account and terminates execution at exactly zero remaining attempts. It also tests entering random characters (like letters or symbols) into numerical.

5.3 Extreme Value/Insufficient Funds Handling:

This testing process forces the code to handle edge cases and logical boundary limits. Testers input negative numbers or zero into the deposit and withdrawal modules to ensure the application's filters reject them. Additionally, a withdrawal request higher than the current balance is executed to verify that the nested conditional loop flags it as an "Insufficient Funds" overdraft attempt and safely blocks the transaction.

5.4 Multi-Transaction Cycle and Session Reset Testing:

This test verifies that the (while True) loop maintains state accuracy over a long, continuous user session. Testers perform a series of operations back-to-back—such as checking the balance, making a deposit, executing a withdrawal, and checking the balance again—to ensure the `account_balance` variable updates dynamically and holds its value correctly across each loop cycle. It also checks that when option 4 is selected, the loop completely breaks, ending the user session safely.

6) FUTURE SCALABILITY ROADMAPS:

This final planning section outlines how the basic ATM simulator can be expanded from a simple text-based training script into a secure, feature-rich, production-ready application.

6.1) File-Based Data Persistence and Database Integration:

Currently, the application loses all data and resets the account balance back to the starting amount whenever the script is closed. To fix this, the roadmap includes connecting the program to file systems (like `.txt` or `.json` files) or database engines (like `SQLite`). This allows the application to permanently save balance updates, retain PIN numbers, and track login details even after the software restarts.

6.2) Multi-User Account Management & Relational Structures:

The current simulator is built for a single user with hardcoded details. Scalability means expanding the system to handle multiple bank customers simultaneously. This upgrade introduces data structures like dictionaries or database tables to securely map unique account numbers to distinct PINs, balances, and customer profiles.

6.3)Graphical User Interface (GUI) Transition:

To move away from the plain command-line interface, the program can be wrapped in a visual layout. By utilizing desktop UI frameworks (such as Python's `Tkinter` or `PyQt`), developers can replace typed commands with an interactive screen featuring digital keypads, functional buttons, and visual popup alerts that closely mimic a real-world ATM screen.

6.4)Advanced Security Controls & Transaction Logs:

To prepare the application for real-world scenarios, future versions will incorporate robust security upgrades. This includes encrypting stored PINs using hashing algorithms, implementing real-time input sanitization to block malicious code injection, and auto-generating a time-stamped text file (a transaction ledger) that logs every deposit and withdrawal for auditing purposes.

7) PROJECT CONCLUSION:

The development and execution of the Simple ATM Simulator successfully demonstrate how fundamental programming building blocks can solve real-world system challenges. By structurally embedding conditional branching (if-elif-else) within a continuous loop (while), the application successfully mimics the operational logic of a real automated banking teller.

Through rigorous test cases, the project shows that a terminal-based system can safely handle sensitive user sessions. The control loops keep the machine running indefinitely across multiple actions, while defensive conditional blocks act as strict guardrails—validating input types, counting down login attempts, and blocking overdraft transactions.

Ultimately, this simulator establishes a clean, predictable, and error-tolerant architectural baseline. While currently operating as a single-session script, the logical layout is highly scalable.